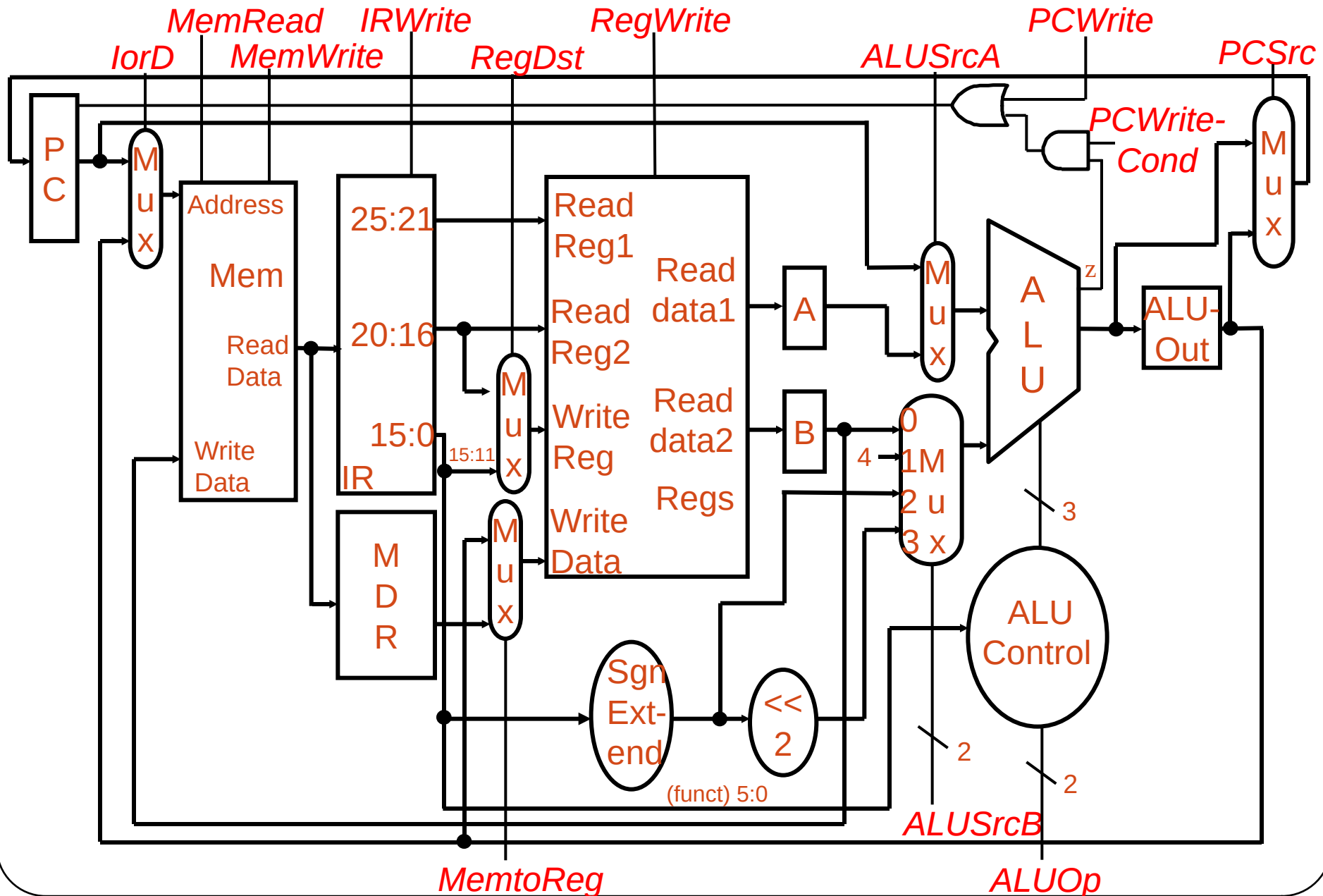


Computer System Architecture

Datapath & Control
(Microarchitecture)

Single Cycle vs. Multi Cycle

Multicycle Datapath + Control Points



Multicycle Instruction Execution

- All instructions execute in 3-5 cycles
 - 3 cycles: **beq**
 - 4 cycles: R-type, **sw**
 - 5 cycles: **lw**
- 1: fetch instruction, $PC=PC+4$
- 2: decode, fetch registers, brnch target
- 3: execute/compute address/branch
- 4: access memory/complete R-type
- 5: (**lw**) store memory

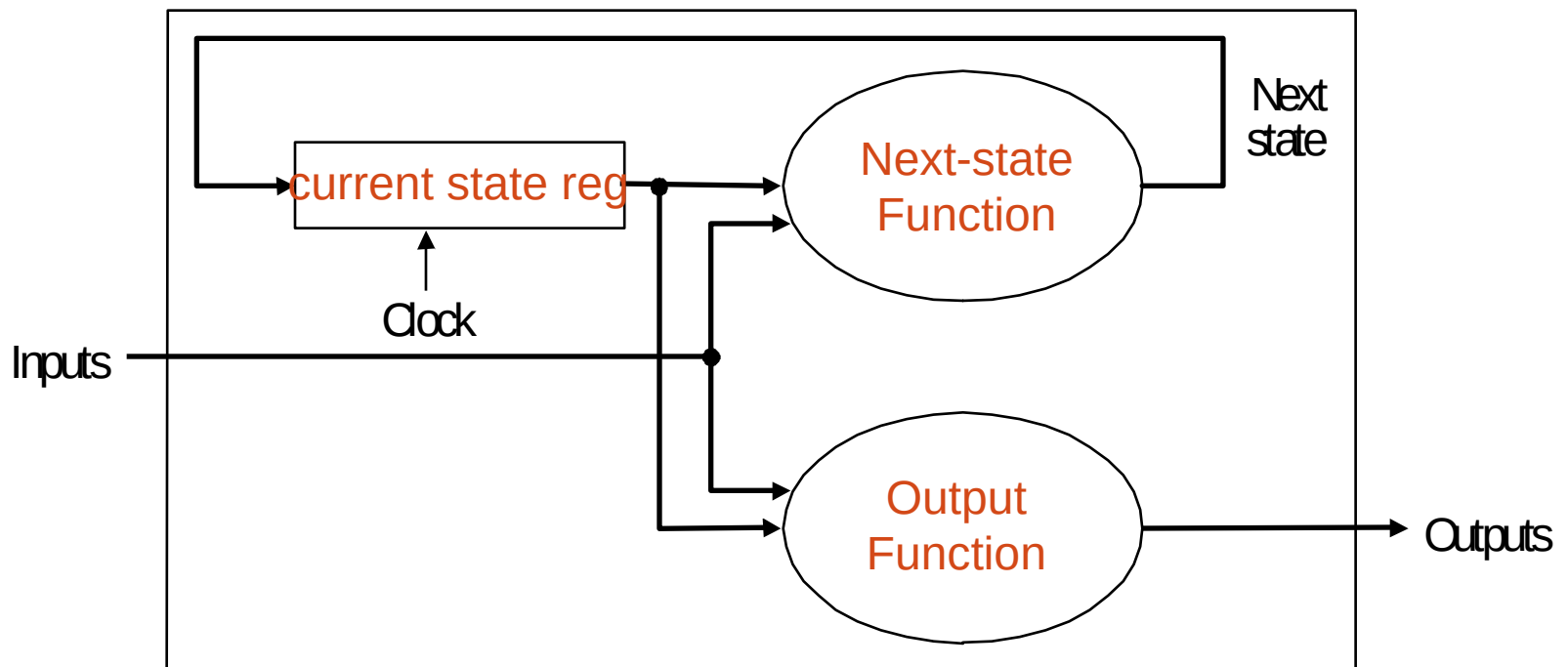
Multicycle – Summary

Step name	Action for R-type instructions	Action for memory-reference instructions	Action for branches	Action for jumps
Instruction fetch	$IR \leftarrow \text{Memory}[PC]$ $PC \leftarrow PC + 4$			
Instruction decode/register fetch	$A \leftarrow \text{Reg}[IR[25:21]]$ $B \leftarrow \text{Reg}[IR[20:16]]$ $ALUOut \leftarrow PC + (\text{sign-extend}(IR[15:0]) \ll 2)$			
Execution, address computation, branch/jump completion	$ALUOut \leftarrow A \text{ op } B$	$ALUOut \leftarrow A + \text{sign-extend}(IR[15:0])$	If $(A == B)$ $PC \leftarrow ALUOut$	$PC \leftarrow \{PC[31:28], (IR[25:0]), 2'b00\}$
Memory access or R-type completion	$\text{Reg}[IR[15:11]] \leftarrow ALUOut$	Load: $MDR \leftarrow \text{Memory}[ALUOut]$ or Store: $\text{Memory}[ALUOut] \leftarrow B$		
Memory read completion		Load: $\text{Reg}[IR[20:16]] \leftarrow MDR$		

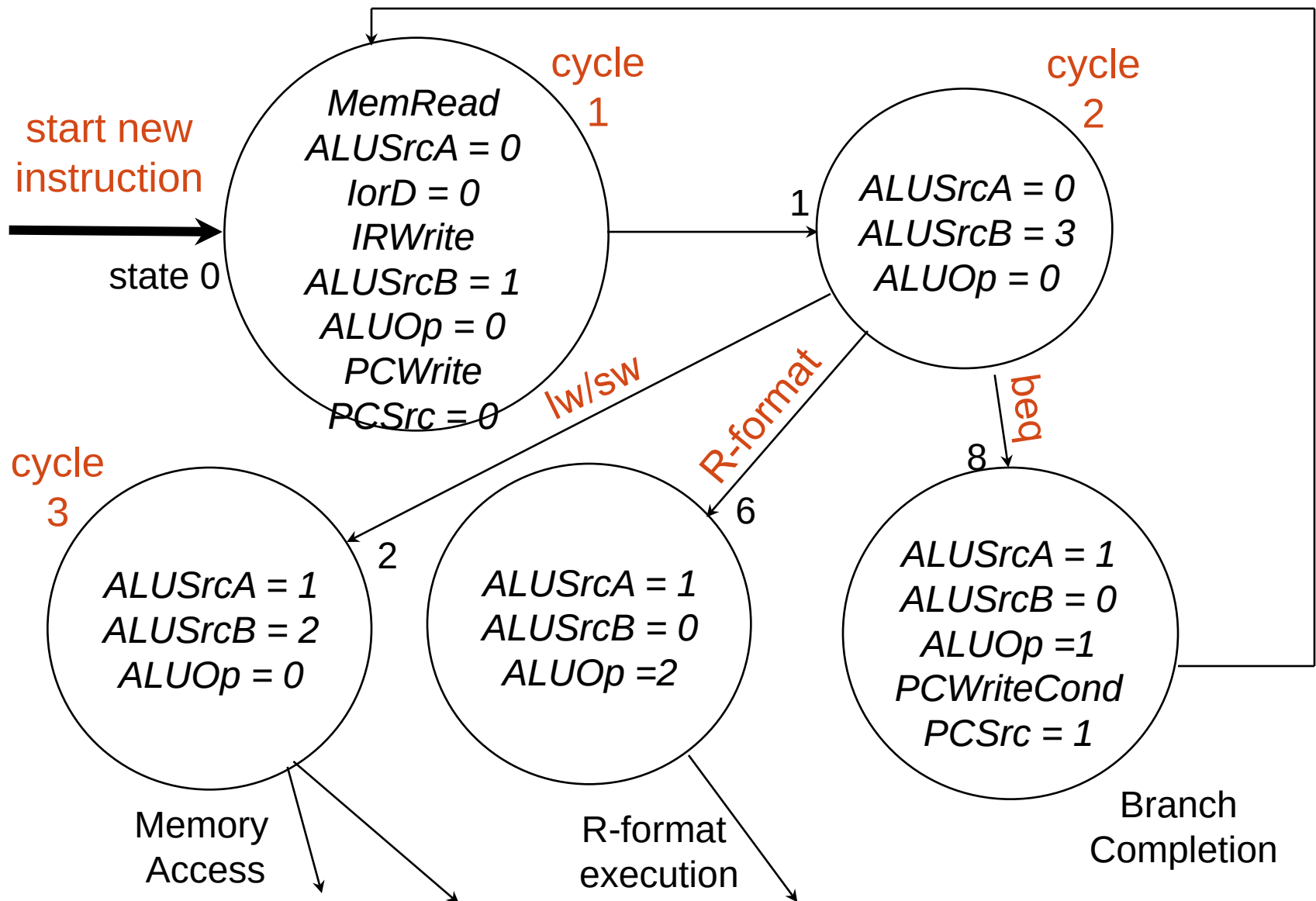
FIGURE 5.30 Summary of the steps taken to execute any instruction class. Instructions take from three to five execution steps. The first two steps are independent of the instruction class. After these steps, an instruction takes from one to three more cycles to complete, depending on the instruction class. The empty entries for the Memory access step or the Memory read completion step indicate that the particular instruction class takes fewer cycles. In a multicycle implementation, a new instruction will be started as soon as the current instruction completes, so these cycles are not idle or wasted. As mentioned earlier, the register file actually reads every cycle, but as long as the IR does not change, the values read from the register file are identical. In particular, the value read into register B during the Instruction decode stage, for a branch or R-type instruction, is the same as the value stored into B during the Execution stage and then used in the Memory access stage for a store word instruction.

Implementing Multicycle Control: Finite State Machine

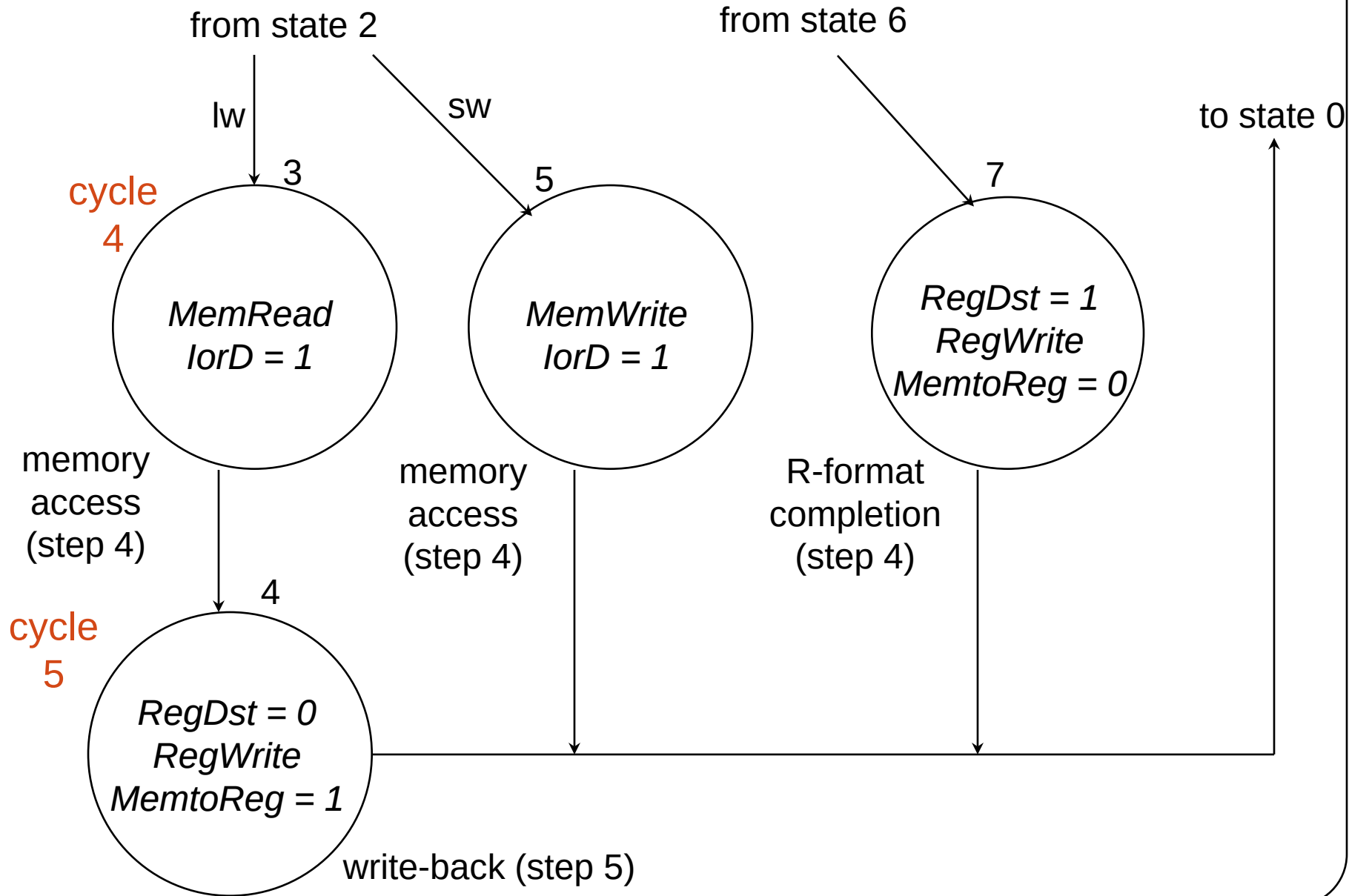
- internal storage (current state register)
- two combinational circuits:
 - next state function; output function



FSM diagram for Multicycle Machine



FSM controller: execution cycles 3-5



Assignment - 2

- How many multicycles will it take to execute this code?

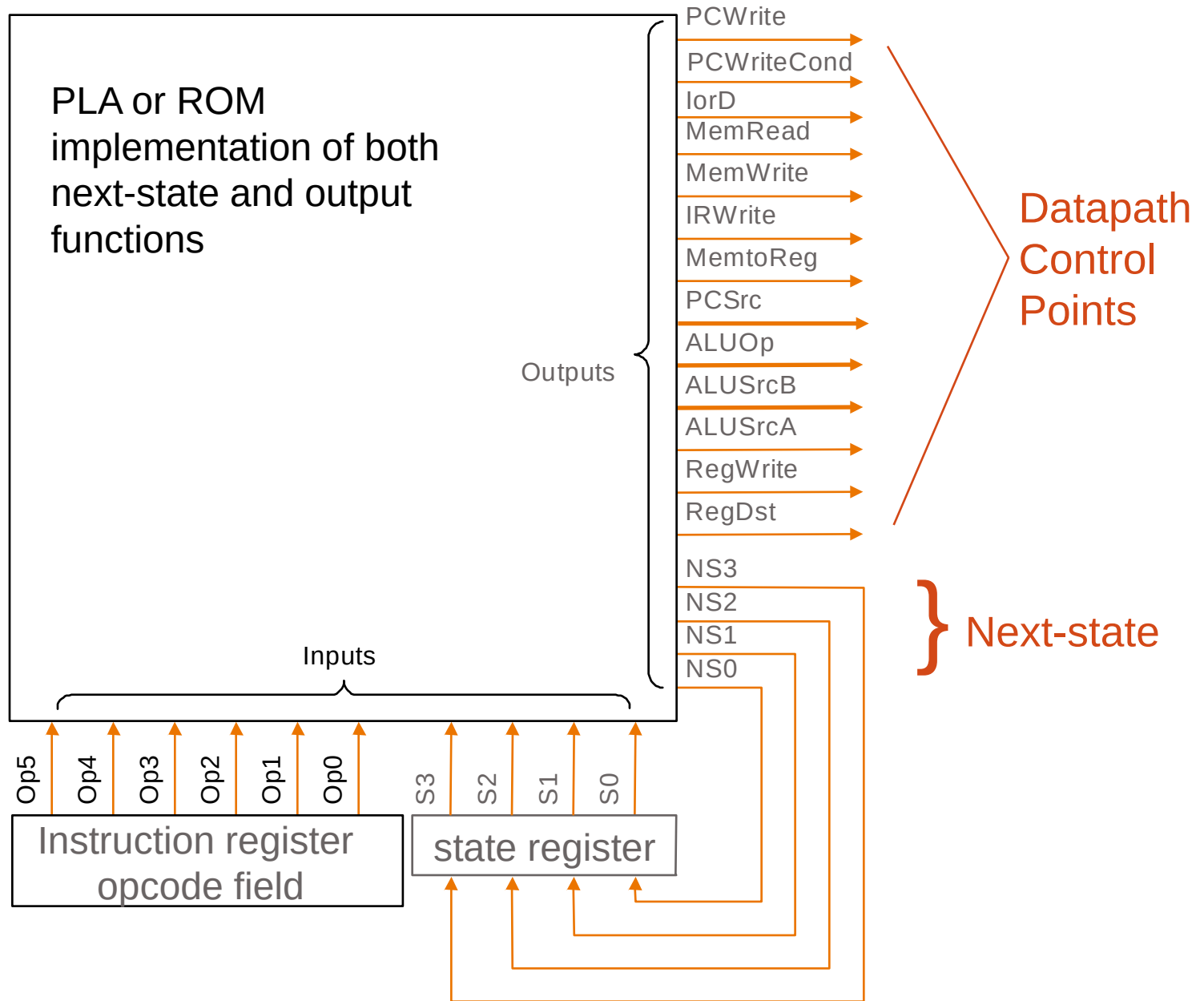
```
lw $t2, 0($t3)
lw $t3, 4($t3)
beq $t2, $t3, Label #assume no jump
add $t5, $t2, $t3
sw $t5, 8($t3)
```

Label: ...

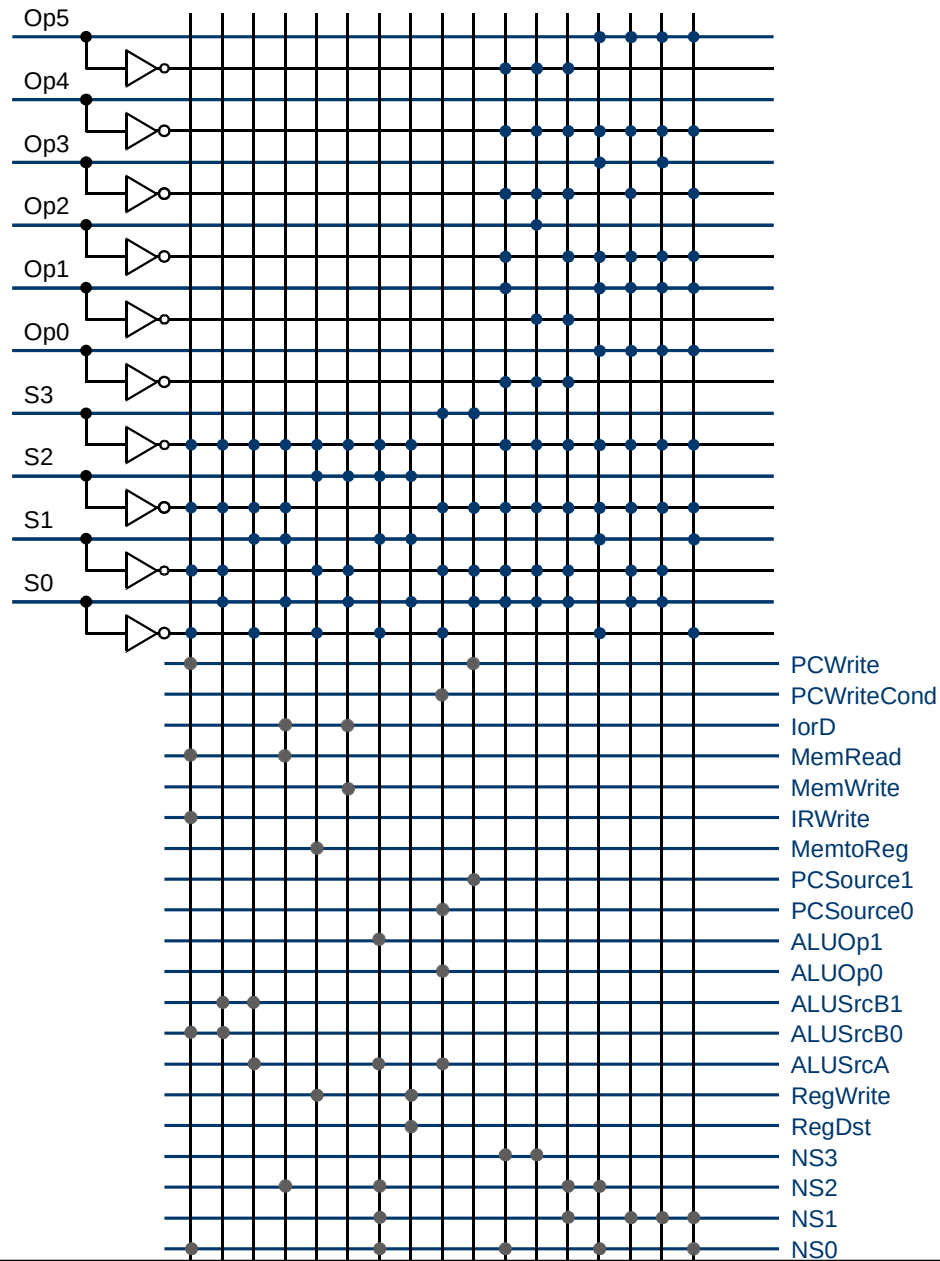
- What is going on during the 8th cycle of execution?
- In what cycle does the actual ALU addition of **\$t2** and **\$t3** takes place?



Implementing the FSM controller

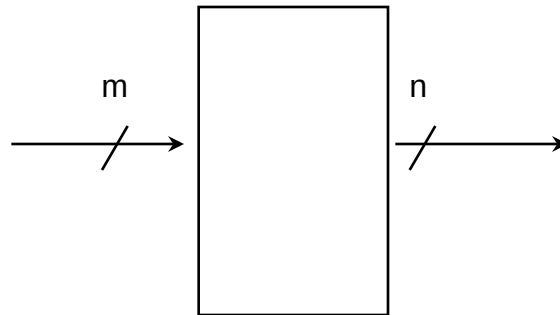


PLA Implementation



ROM Implementation

- ROM = "Read Only Memory"
 - values of memory locations are fixed ahead of time
- A ROM can be used to implement a truth table
 - if the address is m -bits, we can address 2^m entries in the ROM.
 - our outputs are the bits of data that the address points to.
 m is the "height", and n is the "width" equal to number of outputs.



ROM Implementation - Optimize

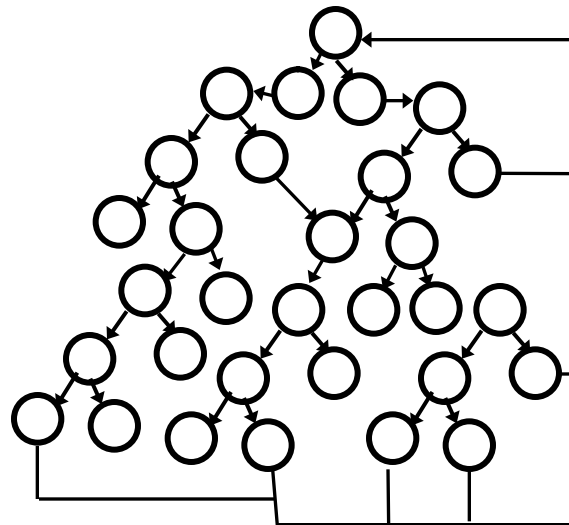
- How many inputs are there?
6 bits for opcode, 4 bits for state = 10 address lines
(i.e., $2^{10} = 1024$ different addresses)
- How many outputs are there?
16 datapath-control outputs, 4 state bits = 20 outputs
- ROM is $2^{10} \times 20 = 20\text{K bits}$ (and a rather unusual size, so go for next size chip)
- Rather wasteful due to lots of don't care situations $\Rightarrow$ the outputs only depend on states, not opcodes.
- ROM: Break up the table into two parts
 - 4 next state bits $\rightarrow$ 16 control outputs, $2^4 \times 16$ bits of ROM
 - 10 bits (state+op) $\rightarrow$ 4 next state bits, $2^{10} \times 4$ bits of ROM
 - Total: 4.3K bits of ROM $\Rightarrow$ Lots of savings.

ROM vs PLA

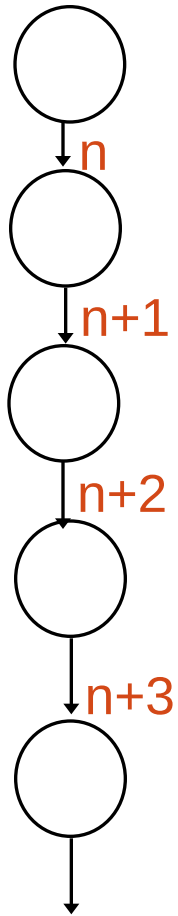
- PLA is much smaller
 - can share product terms
 - only need entries that produce an active output
 - can take into account don't cares
- Size is $(\text{\#inputs} \times \text{\#product-terms}) + (\text{\#outputs} \times \text{\#product-terms})$
- For this example $= (10 \times 17) + (20 \times 17) = 510$ PLA cells
PLA cells usually about the size of a ROM cell (slightly bigger)

Alternative to FSM for Multi-cycle?

- MIPS-lite has (about) 7 instructions, 10 FSM states
- Real machines have 100 or more instructions; real controllers have hundreds, or even thousands of states!
- Problem: FSM Bubble-diagram too large
- Instead of **Hardware Implementation** (FSM) ??



Observation about real machines



- **Machine Language**: next instruction to be executed is usually implied
 - PC register determines instruction
 - next instruction always at PC+4 (unless branch or jump)
- FSM Controller: **often only one exit arc** from current state to next state
- Suppose **borrow idea from Machine Language**, represent each control step as some kind of “instruction”?
- Leads to **Microprogrammed Control**

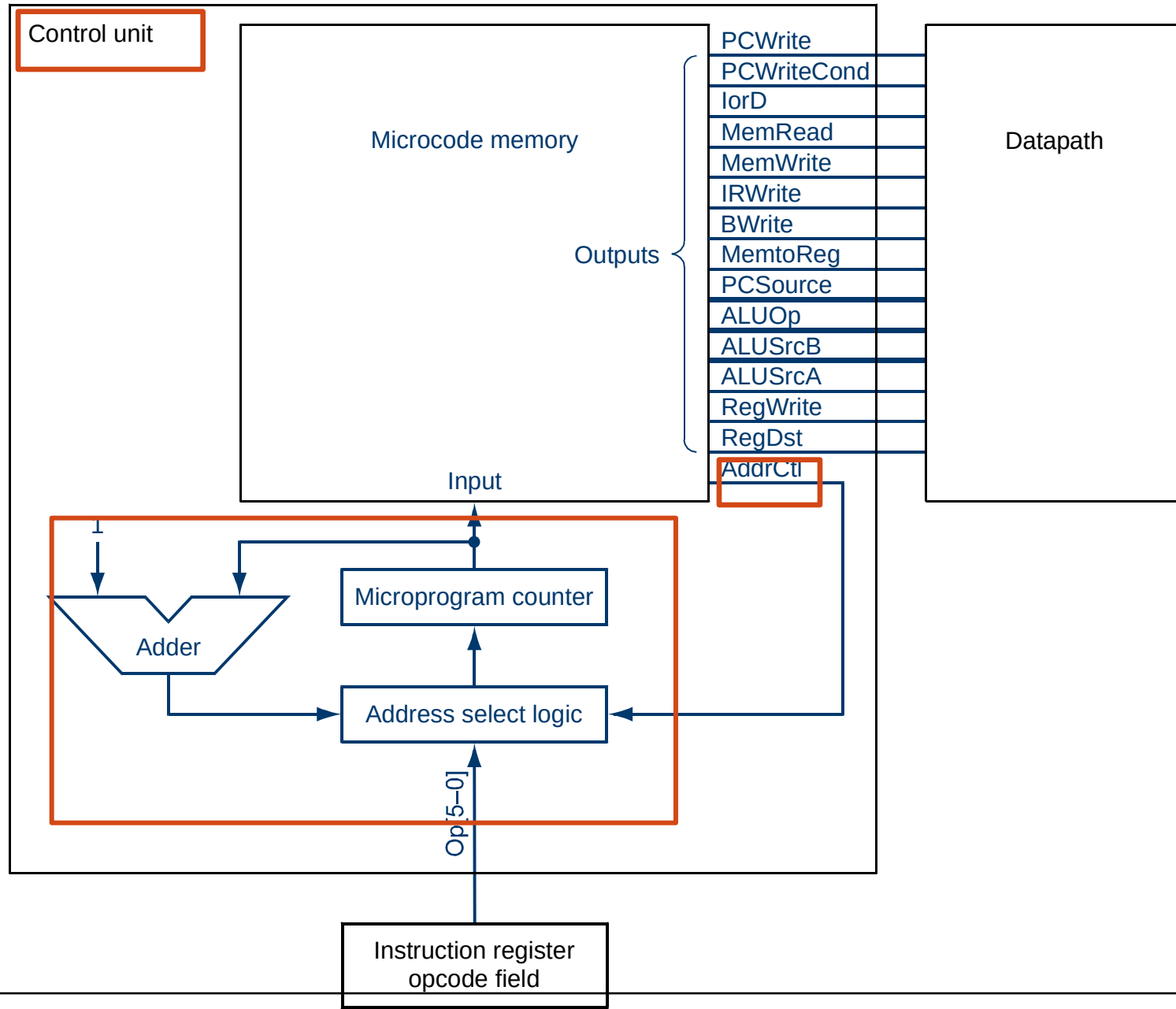
Micro-programmed Control

- In microprogrammed control, FSM states become microinstructions of a microprogram (“microcode”)
 - one FSM state=one microinstruction
 - usually represent each micro-instruction textually, like an assembly instruction
- FSM current state register becomes the microprogram counter (micro-PC)
 - **normal sequencing**: add 1 to micro-PC to get next micro-instruction
 - **microprogram branch**: separate logic determines next microinstruction

Microprogramming Vs Hardwired Control

- **Microprogramming offers flexibility** for design and architectural changes.
 - The control memory (ROM) can be reprogrammed or replaced. Hardwired control is difficult to design for complex set architecture. Once it is designed, no further change is possible
- **Microprogramming is slow** because the control memory is accessed in every cycle. Memory access is slow.
 - Hardwired control is fast because the cycle time depends on the **combinational logic delay** of the control unit, which is much less than **memory access time**.

Microprogramming



Microprogramming

- A specification methodology
 - appropriate if hundreds of opcodes, modes, cycles, etc.
 - signals specified **symbolically** using **microinstructions**

Label	ALU control	SRC1	SRC2	Register control	Memory	PCWrite control	Sequencing
Fetch	Add	PC	4		Read PC	ALU	Seq
	Add	PC	Extshft	Read			Dispatch 1
Mem1	Add	A	Extend				Dispatch 2
LW2					Read ALU		Seq
				Write MDR			Fetch
SW2					Write ALU		Fetch
Rformat1	Func code	A	B				Seq
				Write ALU			Fetch
BEQ1	Subt	A	B			ALUOut-cond	Fetch
JUMP1						Jump address	Fetch

Will two implementations of the same architecture have the same microcode? What would a microassembler do?

Microinstruction format

Field name	Value	Signals active	Comment
ALU control	Add	ALUOp = 00	Cause the ALU to add.
	Subt	ALUOp = 01	Cause the ALU to subtract; this implements the compare for branches.
	Func code	ALUOp = 10	Use the instruction's function code to determine ALU control.
SRC1	PC	ALUSrcA = 0	Use the PC as the first ALU input.
	A	ALUSrcA = 1	Register A is the first ALU input.
SRC2	B	ALUSrcB = 00	Register B is the second ALU input.
	4	ALUSrcB = 01	Use 4 as the second ALU input.
	Extend	ALUSrcB = 10	Use output of the sign extension unit as the second ALU input.
	Extshft	ALUSrcB = 11	Use the output of the shift-by-two unit as the second ALU input.
Register control	Read		Read two registers using the rs and rt fields of the IR as the register numbers and putting the data into registers A and B.
	Write ALU	RegWrite, RegDst = 1, MemtoReg = 0	Write a register using the rd field of the IR as the register number and the contents of the ALUOut as the data.
	Write MDR	RegWrite, RegDst = 0, MemtoReg = 1	Write a register using the rt field of the IR as the register number and the contents of the MDR as the data.
Memory	Read PC	MemRead, lorD = 0	Read memory using the PC as address; write result into IR (and the MDR).
	Read ALU	MemRead, lorD = 1	Read memory using the ALUOut as address; write result into MDR.
	Write ALU	MemWrite, lorD = 1	Write memory using the ALUOut as address, contents of B as the data.
PC write control	ALU	PCSource = 00 PCWrite	Write the output of the ALU into the PC.
	ALUOut-cond	PCSource = 01, PCWriteCond	If the Zero output of the ALU is active, write the PC with the contents of the register ALUOut.
	jump address	PCSource = 10, PCWrite	Write the PC with the jump address from the instruction.
Sequencing	Seq	AddrCtl = 11	Choose the next microinstruction sequentially.
	Fetch	AddrCtl = 00	Go to the first microinstruction to begin a new instruction.
	Dispatch 1	AddrCtl = 01	Dispatch using the ROM 1.
	Dispatch 2	AddrCtl = 10	Dispatch using the ROM 2.

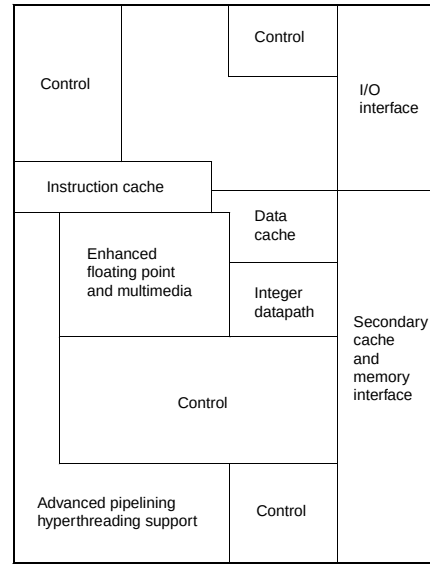
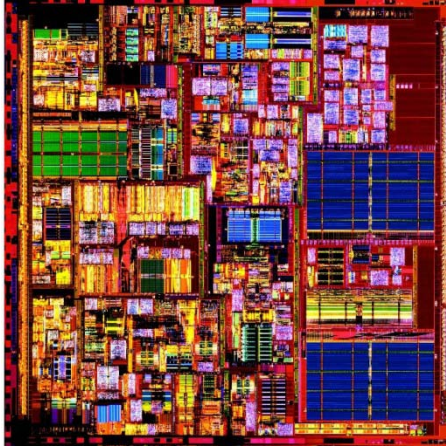
Microcode: Trade-offs

- Distinction between specification and implementation is sometimes blurred
- Specification Advantages:
 - Easy to design and write
 - Design architecture and microcode in parallel
- Implementation (ROM) Advantages
 - Easy to change since values are in memory
 - Can emulate other architectures
 - Can make use of internal registers

Historical Perspective - Microprogramming

- In the '60s and '70s microprogramming was very important for implementing machines
- This led to more sophisticated ISAs and the VAX
- In the '80s RISC processors based on pipelining became popular
- Pipelining the microinstructions is also possible!
- Implementations of IA-32 architecture processors since 486 use:
 - “**hardwired control**” for simpler instructions
(few cycles, FSM control implemented using PLA or ROM)
 - “**microcoded control**” for more complex instructions
(large numbers of cycles, central control store)

Pentium 4



- Somewhere in all that “control we must handle complex instructions
- Processor executes simple microinstructions, 70 bits wide (hardwired)
- 120 control lines for integer datapath (400 for floating point)
- If an instruction requires more than 4 microinstructions to implement,
control from microcode ROM (8000 microinstructions)
- Its complicated!

Microarchitecture - Summary

- If we understand the **instructions (ISA)**...

We can build a simple processor!

- If instructions take different amounts of time, **multi-cycle** is better
- **Datapath** implemented using:
 - Combinational logic modules (ALU) for arithmetic
 - State holding elements (Reg, Mem) to remember bits
- **Control** implemented using:
 - Combinational logic for single-cycle implementation
 - Finite state machine (hardware) or Microprogramming for multi-cycle implementation

Pipelining (Next Topic)

- Techniques illustrated so far in **Micro-architecture** are at the heart of every computer
- All recent computers, however, go beyond these techniques, and use **pipelining** to improve performance
- By overlapping execution of multiple instructions, pipelining can achieve:
 - *throughput close to 1 instruction* per clock cycle (*like single-cycle machine*)
 - with a **clock cycle time** determined by the delay of **individual datapath** components (*like multi-cycle machine*)